

1. Equivalencia de modelos de cálculo

Teorema 1. *Los siguientes hechos son equivalentes:*

1. *f es parcialmente calculable por una MT.*
2. *f es parcialmente calculable por un programa Post-Turing.*
3. *f es parcialmente calculable por un programa con variables.*

Análogamente, los siguientes hechos son equivalentes:

1. *L es aceptado por una MT.*
2. *L es aceptado por un programa Post-Turing.*
3. *L es aceptado por un programa con variables.*

Demostración. Se basa en tres simulaciones circulares: (1) Programa con variables $\rightarrow$ Programa Post-Turing, (2) Programa Post-Turing $\rightarrow$ MT, (3) MT $\rightarrow$ Programa con variables. $\square$

2. Programa con variables $\rightarrow$ Programa Post-Turing

Idea. Sean las variables $V_1, \dots, V_l$ (incluyendo las de entrada $X_1, \dots, X_m$, las de trabajo $Z_1, \dots, Z_k$ y la de salida Y , con $l = m + k + 1$). El programa Post-Turing almacena todas las variables en la cinta con la disposición:

$$\# V_1 \# \dots \# V_j \# \dots \# V_l \#$$

Al principio de cada instrucción simulada, el cabezal está en el blanco justo a la izquierda de V_1 .

Macros auxiliares.

- **RIGHT TO NEXT BLANK:** avanza a la derecha hasta encontrar $\#$.
- **LEFT TO NEXT BLANK:** avanza a la izquierda hasta encontrar $\#$.
- **MOVE BLOCK RIGHT:** desplaza un bloque de símbolos una posición a la derecha (recorre el bloque de derecha a izquierda, copiando cada símbolo una posición a la derecha, y escribe $\#$ al inicio).
- **ERASE BLOCK:** borra un bloque escribiendo $\#$ en cada casilla hasta encontrar otro $\#$.

Simulación de cada instrucción:

1. $V_j \leftarrow a_i V_j$ (añadir símbolo al principio):
 - a) **RIGHT TO NEXT BLANK** $[l]$ veces $\rightarrow$ llega al final.
 - b) **MOVE BLOCK RIGHT** $[l - j + 1]$ veces $\rightarrow$ abre un hueco justo antes de V_j .
 - c) **RIGHT; PRINT** $a_i \rightarrow$ escribe el símbolo nuevo.
 - d) **LEFT TO NEXT BLANK** $[j]$ veces $\rightarrow$ vuelve al inicio.
2. $V_j \leftarrow V_j^-$ (eliminar último símbolo):
 - a) **RIGHT TO NEXT BLANK** $[j]$ veces; **LEFT** $\rightarrow$ se sitúa en el último símbolo de V_j .
 - b) Si es $\#$, V_j ya es vacía: salta a $[C]$ y vuelve al inicio.
 - c) Si no, borra ese símbolo desplazando los bloques restantes a la izquierda con **MOVE BLOCK RIGHT** $[j]$ veces (sobre lo que queda a la derecha), luego **RIGHT**, y vuelve al inicio con **LEFT TO NEXT BLANK** $[j - 1]$.
3. **IF** V_j **ENDS** a_i **GOTO** L:
 - a) **RIGHT TO NEXT BLANK** $[j]$ veces; **LEFT** $\rightarrow$ lee el último símbolo de V_j .
 - b) Si es a_i , vuelve al inicio con **LEFT TO NEXT BLANK** $[j]$ y salta a L.
 - c) Si no es a_i , avanza con **RIGHT** y vuelve al inicio con **LEFT TO NEXT BLANK** $[j]$.
4. **HALT:** se simula borrando la entrada y las variables intermedias con **ERASE BLOCK** $[l - 1]$ y parando. Si solo se quiere aceptar un lenguaje, no hace falta borrar. $\square$

3. Programa Post-Turing $\rightarrow$ MT

Construcción. La MT tendrá los mismos alfabetos de entrada y de trabajo que el programa Post-Turing. Sea k el número de instrucciones del programa. La MT tiene un estado q_i por cada instrucción I_i , más un estado final q_f y un estado q_{k+1} sin transiciones. El estado inicial corresponde a la primera instrucción.

Traducción de instrucciones:

- **PRINT** a_i : $\delta(q_i, a) = (q_{i+1}, a_i, S)$ para todo $a \in B$.
- **RIGHT:** $\delta(q_i, a) = (q_{i+1}, a, D)$ para todo $a \in B$.

- LEFT: $\delta(q_i, a) = (q_{i+1}, a, I)$ para todo $a \in B$.
- IF a_k GOTO I_j : $\delta(q_i, a_k) = (q_j, a_k, S)$ y $\delta(q_i, a) = (q_{i+1}, a, S)$ si $a \neq a_k$.
- HALT: $\delta(q_i, a) = (q_f, a, S)$ para todo $a \in B$.

Con estas transiciones la MT funciona exactamente igual que el programa Post-Turing. $\square$

4. MT $\rightarrow$ Programa con variables

Idea. Sea $B = \{a_1, \dots, a_n\}$ el alfabeto de trabajo (incluyendo $\#$). El programa usa tres variables básicas X , Z , Y :

- X : contenido a la izquierda del cabezal.
- Z : símbolo bajo el cabezal.
- Y : contenido a la derecha del cabezal.

Macros auxiliares.

- $V \leftarrow -V$ (eliminar primer símbolo): se copia V en una variable auxiliar U símbolo a símbolo, y después se copia U en V excepto el último símbolo (que era el primero de V).
- $V \leftarrow Va_j$ (añadir al final): se copia a_j en U , se añade V a U símbolo a símbolo (a_j queda al final de U), y se copia U en V .
- IF V STARTS a_i GOTO L : se copia V en U símbolo a símbolo hasta ver el primero; se recopia U en V y se ejecuta la acción correspondiente a ese símbolo.

Inicialización. Toda la entrada está en X . Se vacía X colocando su primer símbolo en Z y el resto en Y (se recorre X símbolo a símbolo, pasando cada uno a Y , y el último extraído queda en Z ; si X es vacío, $Z = \#$).

Simulación de transiciones. A cada estado q_i se le asocia una etiqueta A_i y a cada par (q_i, a_j) una etiqueta B_{ij} .

- En A_i (estados no finales): IF Z ENDS a_j GOTO B_{ij} para $j = 1, \dots, n$.
- En A_i (estados finales): se concatena Z e Y en Y , se añade X al principio de Y símbolo a símbolo, y HALT.

Movimiento a la derecha: si $\delta(q_i, a_j) = (q_m, a_k, D)$:

1. $X \leftarrow Xa_k$ (añadir a_k al final de X).
2. $Z \leftarrow \varepsilon$.
3. Si Y empieza por a_l : $Y \leftarrow -Y$; $Z \leftarrow a_l Z$; ir a A_m .
4. Si Y es vacío: $Z \leftarrow \#Z$; ir a A_m .

Movimiento a la izquierda: si $\delta(q_i, a_j) = (q_m, a_k, I)$:

1. $Y \leftarrow a_k Y$ (añadir a_k al principio de Y).
2. $Z \leftarrow \varepsilon$.
3. Si X termina en a_l : $X \leftarrow X^-$; $Z \leftarrow a_l Z$; ir a A_m .
4. Si X es vacío: $Z \leftarrow \#Z$; ir a A_m . $\square$

5. Equivalencia: cálculo con números

Teorema 2. Una función $f : \mathbb{N} \rightarrow \mathbb{N}$ es parcialmente calculable por un programa con variables numéricas si y solo si es parcialmente calculable utilizando una codificación con palabras.

Idea de la demostración. Se basa en simular las instrucciones de un programa con números ($A \leftarrow A + 1$, $A \leftarrow A - 1$, IF $A \neq 0$ GOTO L) mediante un programa con cadenas que trabaje con las codificaciones $C(n)$ de esos números en un alfabeto, y recíprocamente. Las simulaciones de sumar 1 y restar 1 sobre codificaciones se han visto como ejemplos en el tema. $\square$